

1 - Programas secuenciales

Alejandro Veloz

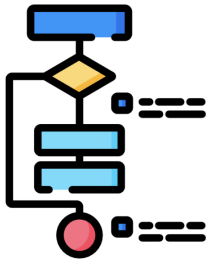
aavelozb.github.io/py



Parte I – Algoritmos

Antes de escribir código, hay que saber *qué* se quiere resolver

Algoritmo



- Es una **secuencia finita** y **precisa** de pasos para resolver un **problema**.
- Un problema es la necesidad de **transformar** un **estado inicial** en un **estado final**, respetando ciertas restricciones.
- El estado inicial contiene los **datos de entrada**.
- El estado final contiene los **datos de salida** (la solución).
- Casi siempre aparecen, además, **datos intermedios**.

¿Cuándo un algoritmo está bien definido?

- **Preciso:** cada paso está descrito sin ambigüedad.

¿Cuándo un algoritmo está bien definido?

- **Preciso:** cada paso está descrito sin ambigüedad.
- **Determinista:** con las mismas entradas, produce siempre las mismas salidas.

¿Cuándo un algoritmo está bien definido?

- **Preciso**: cada paso está descrito sin ambigüedad.
- **Determinista**: con las mismas entradas, produce siempre las mismas salidas.
- **Finito**: termina después de un número finito de pasos.

¿Cuándo un algoritmo está bien definido?

- **Preciso**: cada paso está descrito sin ambigüedad.
- **Determinista**: con las mismas entradas, produce siempre las mismas salidas.
- **Finito**: termina después de un número finito de pasos.
- **Correcto**: entrega efectivamente la solución del problema planteado.

¿Cuándo un algoritmo está bien definido?

- **Preciso**: cada paso está descrito sin ambigüedad.
- **Determinista**: con las mismas entradas, produce siempre las mismas salidas.
- **Finito**: termina después de un número finito de pasos.
- **Correcto**: entrega efectivamente la solución del problema planteado.

Recién después de todo esto nos preocupamos de que sea **eficiente**.

Ejemplos de problemas

- Dado un conjunto de números, determinar los números ordenados de menor a mayor.
- Dado un conjunto de ciudades, encontrar el camino más corto que las recorre.
- Dado un mensaje de correo, encontrar la probabilidad de que sea spam.
- Dadas las longitudes de los lados de un triángulo, calcular su área.
- ...

El problema de la ambigüedad



Mi madre dice:

“Cariño, anda al súper y me traes una botella de leche, y si tienen huevos, compra seis.”

El problema de la ambigüedad



Mi madre dice:

“Cariño, anda al súper y me traes una botella de leche, y si tienen huevos, compra seis.”

Cuando vuelvo me pregunta:

“¿Por qué trajiste seis botellas de leche?”

El problema de la ambigüedad



Mi madre dice:

“Cariño, anda al súper y me traes una botella de leche, y si tienen huevos, compra seis.”

Cuando vuelvo me pregunta:

“¿Por qué trajiste seis botellas de leche?”

Respondo:

“Porque había huevos.”

Comprensión del problema

Para poder resolver un problema hay que **comprenderlo bien** y ser capaz de **formalizarlo** de alguna manera.

Comprensión del problema

Para poder resolver un problema hay que **comprenderlo bien** y ser capaz de **formalizarlo** de alguna manera.

De lo contrario, corremos el riesgo de **resolver muy bien el problema equivocado**.

Comprensión del problema

Para poder resolver un problema hay que **comprenderlo bien** y ser capaz de **formalizarlo** de alguna manera.

De lo contrario, corremos el riesgo de **resolver muy bien el problema equivocado**.

Al formalizar definimos el **estado final** al que queremos llegar: hay que imaginar cómo se vería una solución.

En esta etapa nos concentramos en el **qué** y no en el **cómo**.

¿Cómo se expresa un algoritmo?

1. **Lenguaje natural:** cómodo, pero ambiguo.
2. **Pseudocódigo:** instrucciones estructuradas, independientes de un lenguaje.
3. **Diagrama de flujo:** representación gráfica del flujo de ejecución.
4. **Lenguajes de bloques** (Scratch, Blockly).

¿Cómo se expresa un algoritmo?

1. **Lenguaje natural:** cómodo, pero ambiguo.
2. **Pseudocódigo:** instrucciones estructuradas, independientes de un lenguaje.
3. **Diagrama de flujo:** representación gráfica del flujo de ejecución.
4. **Lenguajes de bloques** (Scratch, Blockly).

Ninguno de ellos es un programa todavía: son formas de **pensar** la solución.

Problema de ejemplo

Determine si un número natural n es **primo** o **compuesto**.

Definición: n es primo si sus únicos divisores son 1 y él mismo. En caso contrario es compuesto.

Problema de ejemplo

Determine si un número natural n es **primo** o **compuesto**.

Definición: n es primo si sus únicos divisores son 1 y él mismo. En caso contrario es compuesto.

1. Lenguaje natural

- Buscar algún valor d entre 2 y $n-1$ que sea divisor de n .
- Si existe al menos uno de esos valores, entonces n es compuesto; en caso contrario, es primo.

2. Pseudocódigo

```
leer n
es_primo = verdadero
d = 2

mientras d sea menor que n:
    si n es divisible por d:
        es_primo = falso
        d = d + 1

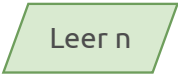
si es_primo es verdadero:
    escribir "n es primo"
si no:
    escribir "n es compuesto"
```

3. Diagramas de flujo: simbología



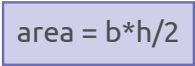
inicio

Terminal: inicio y fin del algoritmo.



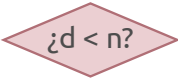
Leer n

Entrada / salida: leer datos, escribir resultados.



$\text{area} = b \cdot h / 2$

Proceso: un cálculo o una asignación.



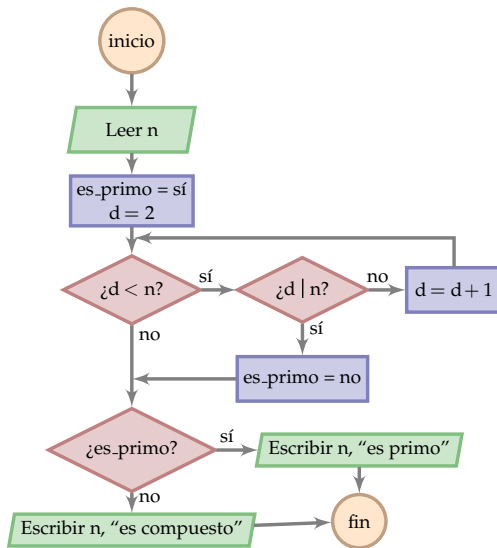
¿d < n?

Decisión: pregunta con salidas *sí* / *no*.



Flujo: orden en que se ejecutan los pasos.

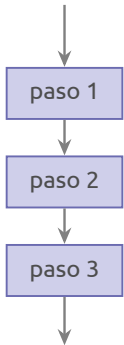
3. Diagrama de flujo del ejemplo



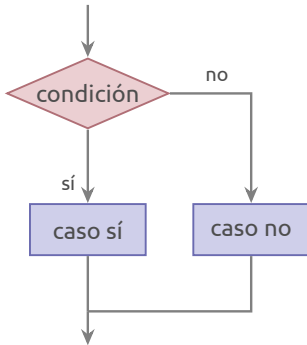
Estructura de un algoritmo

Las instrucciones de un algoritmo se combinan de sólo tres maneras:

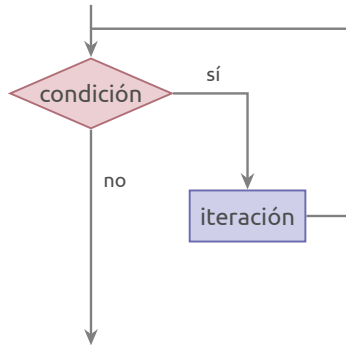
Secuencial



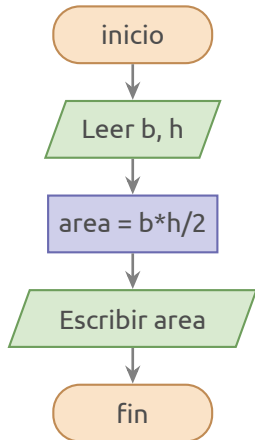
Decisión



Repetición



En esta clase: sólo instrucciones secuenciales



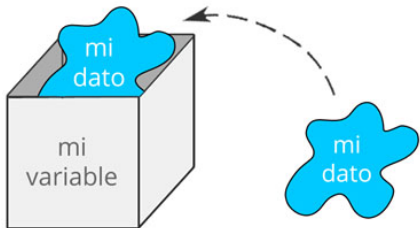
Cada bloque del diagrama se traduce, casi línea a línea, a Python:

```
b = float(input('base: '))
h = float(input('altura: '))
area = b * h / 2
print('area =', area)
```

Las instrucciones se ejecutan **una después de la otra**, de arriba hacia abajo, **exactamente una vez cada una**.

Parte II – Programas secuenciales en Python

Variables



Una **variable** es un espacio en memoria, identificado por un **nombre**, que guarda un **valor** que puede cambiar durante la ejecución del programa.

La variable se crea al asignarle un valor por primera vez:

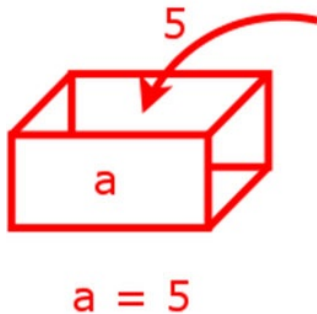
```
x = 5  
nombre = "Juan"
```

Asignación de variables

Una asignación tiene la forma:

`<variable> = <expresión>`

1. Primero se **evalúa la expresión** a la derecha del signo =.
2. El resultado se **asigna a la variable** a la izquierda.



Por eso $x = x + 1$ tiene sentido en Python, aunque no lo tenga en matemáticas.

Tipos de variables

- `int` (enteros): 5, 2026.
- `float` (reales): 1.414213562, 6.02e+23.
- `bool` (lógicos o booleanos): `True`, `False`.
- `str` (textos, cadena de caracteres): `'Hola'`, `"Me encanta Python"`.

Operadores

Operación	Símbolo	Ejemplo
Suma	+	$10 + 5 \rightarrow 15$
Resta	-	$3 - 7 \rightarrow -4$
Multiplicación	*	$3 * 9 \rightarrow 27$
División	/	$7 / 2 \rightarrow 3.5$
División entera	//	$7 // 2 \rightarrow 3$
Módulo (resto de la división entera)	%	$7 \% 2 \rightarrow 1$
Potencia	**	$2 * * 3 \rightarrow 8$

Precedencia de operadores

Las expresiones se evalúan siguiendo reglas de precedencia.

De mayor a menor, la precedencia de operadores es la siguiente:

1. (,) (parentesis)
2. abs(), sqrt(), randint() (llamada a funciones)
3. **
4. $+x$, $-x$ (unario)
5. *, /, //, %
6. +, -

¿Cuál es el resultado en Python?

5 + 3 * 2

(5 + 3) * 2

10 / 2 + 3 ** 2

10 - 4 + 2

2 ** 3 ** 2

(2 ** 3) ** 2

¿Cuál es el resultado en Python?

`5 + 3 * 2`

`(5 + 3) * 2`

`10 / 2 + 3 ** 2`

`10 - 4 + 2`

`2 ** 3 ** 2`

`(2 ** 3) ** 2`

11

16

14.0

8

512

64

¿Cuál es el resultado en Python?

5 + 3 * 2

(5 + 3) * 2

10 / 2 + 3 ** 2

10 - 4 + 2

2 ** 3 ** 2

(2 ** 3) ** 2

11

16

14.0

8

512

64

Ojo con los dos últimos: ** asocia por la **derecha**, mientras que +, -, *, / y // asocian por la **izquierda**.

¿Qué muestran estos programas?

```
edad = 25  
nombre = "Kiwi"  
edad = 30  
resultado = edad * 2  
print(resultado)
```

¿Qué muestran estos programas?

```
edad = 25  
nombre = "Kiwi"  
edad = 30  
resultado = edad * 2  
print(resultado)
```

60

```
x = 10  
x = x * 10  
print(x)
```

¿Qué muestran estos programas?

```
edad = 25  
nombre = "Kiwi"  
edad = 30  
resultado = edad * 2  
print(resultado)
```

60

```
x = 10  
x = x * 10  
print(x)
```

100

¿Qué muestran estos programas?

```
edad = 25  
nombre = "Kiwi"  
edad = 30  
resultado = edad * 2  
print(resultado)
```

60

```
x = 10  
x = x * 10  
print(x)
```

100

nombre nunca se usa, y la primera edad se pierde al reasignarla.

Entrada de datos: input()

input() **detiene** el programa, espera que la persona escriba algo y presione Enter, y entrega lo escrito.

```
nombre = input()
```

Entrada de datos: input()

input() **detiene** el programa, espera que la persona escriba algo y presione Enter, y entrega lo escrito.

```
nombre = input()
```

Casi siempre se le pasa un mensaje, que se muestra antes de esperar:

```
nombre = input('Ingrese su nombre: ')
```


Entrada de datos: input()

input() **detiene** el programa, espera que la persona escriba algo y presione Enter, y entrega lo escrito.

```
nombre = input()
```

Casi siempre se le pasa un mensaje, que se muestra antes de esperar:

```
nombre = input('Ingrese su nombre: ')
```

input() entrega SIEMPRE un str, aunque se escriban dígitos.

input() siempre entrega texto

```
a = input('n = ')\nb = a + 10
```

input() siempre entrega texto

```
a = input('n = ')\nb = a + 10
```

TypeError: can only concatenate str (not "int") to str

Peor aún, si los dos son texto **no** hay error, pero tampoco hay suma:

```
'5' + '3'      # entrega '53', no 8
```

input() siempre entrega texto

```
a = input('n = ')\nb = a + 10
```

TypeError: can only concatenate str (not "int") to str

Peor aún, si los dos son texto **no** hay error, pero tampoco hay suma:

```
'5' + '3'      # entrega '53', no 8
```

Convertir apenas se recibe

Por eso el resultado de `input()` se convierte de inmediato:

```
a = int(input('n = '))  
b = a + 10  
print(b)
```

Convertir apenas se recibe

Por eso el resultado de `input()` se convierte de inmediato:

```
a = int(input('n = '))  
b = a + 10  
print(b)
```

- `int(...)` para enteros, `float(...)` para reales.
- Si la persona escribe algo que no es un número, `int()` da error. Está bien: es preferible fallar de inmediato y no arrastrar un dato malo.

Salida de datos: print()

print() recibe varios valores **separados por coma** y los muestra en pantalla:

```
a = 6
x = a**2
print(a, 'al cuadrado es', x)
```

Salida de datos: print()

print() recibe varios valores **separados por coma** y los muestra en pantalla:

```
a = 6
x = a**2
print(a, 'al cuadrado es', x)
```

```
6 al cuadrado es 36
```


Salida de datos: print()

print() recibe varios valores **separados por coma** y los muestra en pantalla:

```
a = 6
x = a**2
print(a, 'al cuadrado es', x)
```

```
6 al cuadrado es 36
```

- La coma agrega **un espacio** entre cada valor.
- Con coma se pueden mezclar textos y números; con + **no** ('El valor es ' + 7 es un error).
- print() sin nada entre paréntesis imprime una **línea en blanco**.

Ejercicio 1

Haga un programa que pida un nombre e imprima un mensaje en el siguiente formato:

```
Ingrese nombre: Alejandro  
Yo soy Alejandro
```

Ejercicio 1

Haga un programa que pida un nombre e imprima un mensaje en el siguiente formato:

```
Ingrese nombre: Alejandro  
Yo soy Alejandro
```

```
nombre = input("Ingrese nombre: ")  
print("Yo soy", nombre)
```

Ejercicio 2

Realice un programa que calcule el área de un rectángulo a partir de las longitudes de sus lados.

Ejercicio 2

Realice un programa que calcule el área de un rectángulo a partir de las longitudes de sus lados.

```
# Programa para calcular el área de un rectángulo  
ancho = 5  
alto = 10  
area = ancho * alto  
print("El área del rectángulo es:", area)
```

Ejercicio 3

Desarrollar un programa que convierta temperatura de Farenheit a Celsius según el ejemplo. La fórmula de conversión es:

$$C = \frac{5}{9} (F - 32)$$

Temp. en Farenheit: 75

El equivalente en Celsius es aproximadamente: 23

Ejercicio 3 - solución

```
f = float(input('Temp. en Farenheit: '))  
c = int( round( (5/9) * (f-32), 2) )  
print('El equivalente en Celsius es aproximadamente:', c)
```

Ejercicio 4

Realice un programa que determine el área de un círculo a partir de su radio.

Ejemplo:

Ingrese el radio de un círculo: 3

El área de la circunferencia es 28.274333882308138

Ejercicio 4 - solución

```
import math

pi = math.pi
radio = float(input('Ingrese el radio de un círculo: '))
print("El área de la circunferencia es", pi*(radio**2))
```

Ejercicio 5 - completar código

Reemplace las partes señaladas con ???

```
lado1 = float(input('Ingrese el primer lado: '))  
??? = float(input('Ingrese el segundo lado: '))  
perimetro = lado1 * ??? + ??? * 2  
area = lado1 ??? lado2  
print('El ??? del rectángulo es:', perimetro)  
print('El área del rectángulo es:', ???)
```

Completar código - solución

Reemplace las partes señaladas con ???

```
lado1 = float(input('Ingrese el primer lado: '))
lado2 = float(input('Ingrese el segundo lado: '))
perimetro = lado1 * 2 + lado2 * 2
area = lado1 * lado2
print('El perímetro del rectángulo es:', perimetro)
print('El área del rectángulo es:', area)
```

Ejercicio 6

Haga un programa que reciba una cantidad de segundos y los convierta al formato horas:minutos:segundos.

Ejemplo:

Cantidad de segundos: 61

0:1:1

Ejercicio 6 - solución

```
segundos = int(input('Cantidad de segundos: '))
horas = segundos // 3600
segundos = segundos % 3600
minutos = segundos // 60
segundos = segundos % 60

tiempo = str(horas) + ':' + str(minutos)
tiempo = tiempo + ':' + str(segundos)
print(tiempo)
```

Ejercicio 6 - solución

```
segundos = int(input('Cantidad de segundos: '))
horas = segundos // 3600
segundos = segundos % 3600
minutos = segundos // 60
segundos = segundos % 60

tiempo = str(horas) + ':' + str(minutos)
tiempo = tiempo + ':' + str(segundos)
print(tiempo)
```

Se usa `str()` para poder **concatenar** con `+`: recuerde que `'texto' + numero` es un error.

Ejercicio 7

Haga un programa que reciba una actual $d2/m2/a2$ y la fecha de nacimiento de una persona $d1/m1/a1$ e indique su edad en años, meses y días. Suponer que todos los años tienen 365 días y que todos los meses tienen 30 días.

Ejemplo:

```
a1: 1990
m1: 3
d1: 1
a2: 2026
m2: 3
d2: 10
print('Su edad es 36 años, 0 meses, 9 días')
```

Ejercicio 7 - solución

```
a1 = int(input('a1: '))
m1 = int(input('m1: '))
d1 = int(input('d1: '))
a2 = int(input('a2: '))
m2 = int(input('m2: '))
d2 = int(input('d2: '))
f1 = a1*365+m1*30+d1
f2 = a2*365+m2*30+d2
dif = f2-f1
a=dif//365
m=(dif%365)//30
d=(dif%365)%30
print('Su edad es ', a, 'años,', m, 'meses,', d, 'días')
```


Ejercicio 8

Un viaje comienza en un punto de coordenadas x_1, y_1 , visita dos puntos x_2, y_2 y x_3, y_3 y vuelve al inicio. Haga un programa que permita determinar la distancia total recorrida.

Ejemplo:

x_1 : 2

y_1 : 3

x_2 : 4

y_2 : 8

x_3 : 3

y_3 : 2

La distancia recorrida es: 12.882140899805819

Ejercicio 8 - solución

```
from math import sqrt

x1 = float(input('x1: '))
y1 = float(input('y1: '))
x2 = float(input('x2: '))
y2 = float(input('y2: '))
x3 = float(input('x3: '))
y3 = float(input('y3: '))
d1 = sqrt ((x1-x2)**2 + (y1-y2)**2)
d2 = sqrt ((x2-x3)**2 + (y2-y3)**2)
d3 = sqrt ((x3-x1)**2 + (y3-y1)**2)
print('La distancia recorrida es: ', d1+d2+d3)
```

Ejercicio 9

Haga un programa que calcule el área de un triángulo a partir de las longitudes de sus lados.

Puede utilizar la fórmula de Herón:

$$A = \sqrt{s(s-a)(s-b)(s-c)},$$

donde a , b y c son las longitudes de cada lado y $s = \frac{a+b+c}{2}$ es el semiperímetro.

Ejemplo:

```
Ingrese longitud de lado 1: 3
Ingrese longitud de lado 2: 3
Ingrese longitud de lado 3: 2
El área del triángulo es 2.8284271247461903
```

Ejercicio 9 - solución

```
l1 = float(input("Ingrese longitud de lado 1: "))
l2 = float(input("Ingrese longitud de lado 2: "))
l3 = float(input("Ingrese longitud de lado 3: "))
s = (l1 + l2 + l3) / 2 # semiperímetro

d1 = s-l1 # diferencia1
d2 = s-l2 # diferencia2
d3 = s-l3 # diferencia3
prod = s*d1*d2*d3 # producto de diferencias y semiperímetro
area = prod ** (1 / 2) # raíz cuadrada
# ¿cómo se podría hacer lo mismo utilizando math.sqrt()?

print("El área del triángulo es", area)
```

Ejercicio 10

Escribir un programa que pida un número de 3 dígitos e imprima la suma de los dígitos del número ingresado.

Ejemplo:

Ingrese un número de 3 dígitos: 245

La suma de los dígitos es 11

Ejercicio 10 - solución

```
n = int(input('Ingrese un número de 3 dígitos: '))  
suma = n%10  
suma += (n//10)%10  
suma += n//100  
print('La suma de los dígitos es', suma)
```

Ejercicio 10 - solución

```
n = int(input('Ingrese un número de 3 dígitos: '))
suma = n%10
suma += (n//10)%10
suma += n//100
print('La suma de los dígitos es', suma)
```

```
n = int(input('Ingrese un número de 3 dígitos: '))
suma = n%10 + (n//10)%10 + n//100
print('La suma de los dígitos es', suma)
```

Ejercicio 10 - solución

```
n = int(input('Ingrese un número de 3 dígitos: '))
suma = n%10
suma += (n//10)%10
suma += n//100
print('La suma de los dígitos es', suma)
```

```
n = int(input('Ingrese un número de 3 dígitos: '))
suma = n%10 + (n//10)%10 + n//100
print('La suma de los dígitos es', suma)
```

```
n = int(input('Ingrese un número de 3 dígitos: '))
print('La suma de los dígitos es', n%10 + (n//10)%10 + n//100)
```


Ejercicio 11

Escribir un programa que pida un número de 4 dígitos e imprima un número con los dígitos invertidos.

Ejemplo:

Ingrese un número de 4 dígitos: 2456

El número invertido es 6542

Ejercicio 11 - solución

```
n = int(input('Ingrese un número de 4 dígitos: '))
n_invertido = 0
ultimo_digito = n%10
n_invertido = ultimo_digito
n = n//10
ultimo_digito = n%10
n_invertido = 10*n_invertido + ultimo_digito
n = n//10
ultimo_digito = n%10
n_invertido = 10*n_invertido + ultimo_digito
n = n//10
ultimo_digito = n%10
n_invertido = 10*n_invertido + ultimo_digito
print('El número invertido es', n_invertido)
```

Ejercicio 12 - ruteo

Rutee el siguiente programa e indique que es lo que imprime.
Cada vez que el valor de una variable cambie, ponga su valor en una **nueva** fila de la tabla.

```
a = 94567
b = 28954
c = 36532
d = 11404
e = 40613
a = a//10000
b = (b//1000)%10
c = (c//100)%10
d = (d//10)%10
e = e%10
print (e,d)
print (c,b,a)
```

a	b	c	d	e

Pantalla:

Ejercicio 12 - solución

```
a = 94567
b = 28954
c = 36532
d = 11404
e = 40613
a = a//10000
b = (b//1000)%10
c = (c//100)%10
d = (d//10)%10
e = e%10
print (e,d)
print (c,b,a)
```

a	b	c	d	e
94567				
	28954			
		36532		
			11404	
				40613
9				
	8			
		5		
			0	
				3

Pantalla:

3 0
5 8 9

Parte III – Más ejercicios

Predecir, depurar, ordenar, completar

Ejercicio 13 - predecir la salida

Sin ejecutar el programa, indique qué imprime cada uno.

```
n = 391
suma = n%10 + (n//10)%10 + n//100
print(suma)
```

Ejercicio 13 - predecir la salida

Sin ejecutar el programa, indique qué imprime cada uno.

```
n = 391
suma = n%10 + (n//10)%10 + n//100
print(suma)
```

13

```
n = 444
suma = n%10 + (n//10)%10 + n//100
resultado = n//suma
print(resultado)
```

Ejercicio 13 - predecir la salida

Sin ejecutar el programa, indique qué imprime cada uno.

```
n = 391
suma = n%10 + (n//10)%10 + n//100
print(suma)
```

13

```
n = 444
suma = n%10 + (n//10)%10 + n//100
resultado = n//suma
print(resultado)
```

37

Ejercicio 13 - predecir la salida

Sin ejecutar el programa, indique qué imprime cada uno.

```
n = 391
suma = n%10 + (n//10)%10 + n//100
print(suma)
```

13

```
n = 444
suma = n%10 + (n//10)%10 + n//100
resultado = n//suma
print(resultado)
```

37

Describe en pocas palabras **qué hace** la expresión suma.

Ejercicio 14 - encontrar el error

Se desea viajar de la ciudad A a la C, pasando por B. El programa pide las distancias y el precio por litro de bencina, y debe mostrar el costo total del viaje. Suponga que un litro permite recorrer una unidad de distancia.

```
distAB = input('Distancia A-B: ')\ndistBC = input('Distancia B-C: ')\nprecio = input('Precio por litro: ')\ncosto = distAB * precio\nprint('El costo total del viaje es: ' + costo)
```

Ejercicio 14 - encontrar el error

Se desea viajar de la ciudad A a la C, pasando por B. El programa pide las distancias y el precio por litro de bencina, y debe mostrar el costo total del viaje. Suponga que un litro permite recorrer una unidad de distancia.

```
distAB = input('Distancia A-B: ')\ndistBC = input('Distancia B-C: ')\nprecio = input('Precio por litro: ')\ncosto = distAB * precio\nprint('El costo total del viaje es: ' + costo)
```

Hay **tres** errores. Corrija uno, vuelva a ejecutar, y aparecerá el siguiente.

Ejercicio 14 - solución

1. Los valores numéricos se están usando como **texto**: falta el `float()` sobre cada `input()`.
2. La fórmula es incorrecta: sólo considera el tramo A-B y olvida B-C.
3. En el `print()` se intenta **sumar** un texto con un número.

Ejercicio 14 - solución

1. Los valores numéricos se están usando como **texto**: falta el `float()` sobre cada `input()`.
2. La fórmula es incorrecta: sólo considera el tramo A-B y olvida B-C.
3. En el `print()` se intenta **sumar** un texto con un número.

```
distAB = float(input('Distancia A-B: '))
distBC = float(input('Distancia B-C: '))
precio = float(input('Precio por litro: '))
costo = (distAB + distBC) * precio
print('El costo total del viaje es:', costo)
```

Ejercicio 15 - encontrar el error

El programa debe indicar cuántos años, meses y días faltan para un evento. Suponga meses de 30 días y años de 360 días.

```
cant_dias = int(input('Cantidad de dias: '))  
a = cant_dias // 360  
m = cant_dias // 30  
d = cant_dias % 30  
print('Faltan', a, 'años,', m, 'meses,', d, 'días')
```

Ejercicio 15 - encontrar el error

El programa debe indicar cuántos años, meses y días faltan para un evento. Suponga meses de 30 días y años de 360 días.

```
cant_dias = int(input('Cantidad de dias: '))  
a = cant_dias // 360  
m = cant_dias // 30  
d = cant_dias % 30  
print('Faltan', a, 'años,', m, 'meses,', d, 'días')
```

Pruebe con `cant_dias = 400`. ¿Qué resultado incorrecto se produce? ¿Dónde está el error?

Ejercicio 15 - solución

Con 400 el programa dice **1 año, 13 meses y 10 días**: los meses se cuentan sobre el total, sin descontar los años ya contados.

```
cant_dias = int(input('Cantidad de días: '))
a = cant_dias // 360
m = (cant_dias % 360) // 30
d = cant_dias % 30
print('Faltan', a, 'años,', m, 'meses,', d, 'días')
```


Ejercicio 15 - solución

Con 400 el programa dice **1 año, 13 meses y 10 días**: los meses se cuentan sobre el total, sin descontar los años ya contados.

```
cant_dias = int(input('Cantidad de días: '))
a = cant_dias // 360
m = (cant_dias % 360) // 30
d = cant_dias % 30
print('Faltan', a, 'años,', m, 'meses,', d, 'días')
```

La línea de **d** **parece** tener el mismo problema, pero es correcta: como 360 es múltiplo de 30, $(cant_dias \% 360) \% 30$ y $cant_dias \% 30$ dan siempre lo mismo.

Ejercicio 16 - ordenar el código

Un capital C a un interés del x por ciento durante n años se convierte en $C(1 + x/100)^n$. El resultado debe redondearse al entero más cercano.

Ordene las líneas. Ejemplo: capital 10000 al 4.5 por ciento se convierte en 24117 al cabo de 20 años.

```
1: x = float(input('Tasa de interés: '))
2: c = c*b
3: print(final)
4: n = int(input('Plazo (años): '))
5: final = round(c)
6: b = a**n
7: a = 1 + x/100
8: c = int(input('Capital: '))
```

Ejercicio 16 - solución

Orden correcto: **8, 1, 4, 7, 6, 2, 5, 3**

```
c = int(input('Capital: '))      # 8
x = float(input('Tasa de interés: ')) # 1
n = int(input('Plazo (años): '))  # 4
a = 1 + x/100                    # 7
b = a**n                        # 6
c = c*b                          # 2
final = round(c)                 # 5
print(final)                     # 3
```

Ejercicio 16 - solución

Orden correcto: **8, 1, 4, 7, 6, 2, 5, 3**

```
c = int(input('Capital: '))           # 8
x = float(input('Tasa de interés: ')) # 1
n = int(input('Plazo (años): '))      # 4
a = 1 + x/100                         # 7
b = a**n                             # 6
c = c*b                               # 2
final = round(c)                      # 5
print(final)                          # 3
```

Primero **todas las entradas**, después los **cálculos** en orden de dependencia, y al final la **salida**.

Ejercicio 17 - completar código

Pedimos rolls de sushi y no sabemos cuántas piezas le tocan a cada uno. Además, según dónde encarguemos, los rolls traen distinta cantidad de piezas.

Reemplace cada ??? y cada ??:

```
cant_rollos    = int(input('Cantidad de rolls: '))
???           = int(input('Cantidad de amigos: '))
piezas_x_roll  = int(input('Piezas por roll: '))

piezas_x_persona = (piezas_x_roll * ???) ?? amigos
piezas_sobrantes = (piezas_x_roll * cant_rollos) ?? amigos

print('Piezas por persona:', ???)
print('y sobran:', ???, 'piezas por repartir')
```

Ejercicio 17 - solución

```
cant_rollos    = int(input('Cantidad de rolls: '))
amigos         = int(input('Cantidad de amigos: '))
piezas_x_roll  = int(input('Piezas por roll: '))

piezas_x_persona = (piezas_x_roll * cant_rollos) // amigos
piezas_sobrantes = (piezas_x_roll * cant_rollos) % amigos

print('Piezas por persona:', piezas_x_persona)
print('y sobran:', piezas_sobrantes, 'piezas por repartir')
```

Ejercicio 17 - solución

```
cant_rollos    = int(input('Cantidad de rolls: '))
amigos         = int(input('Cantidad de amigos: '))
piezas_x_roll  = int(input('Piezas por roll: '))

piezas_x_persona = (piezas_x_roll * cant_rollos) // amigos
piezas_sobrantes = (piezas_x_roll * cant_rollos) % amigos

print('Piezas por persona:', piezas_x_persona)
print('y sobran:', piezas_sobrantes, 'piezas por repartir')
```

Con 6 rolls de 8 piezas entre 5 amigos: **9 piezas** por persona y **sobran 3**.

Aquí // y % responden las dos mitades de la misma pregunta.

Ejercicio 18 - encontrar el error

El programa debería calcular el n-ésimo número de Fibonacci con la fórmula de Binet:

$$F_n = \frac{1}{\sqrt{5}} \left[\left(\frac{1 + \sqrt{5}}{2} \right)^n - \left(\frac{1 - \sqrt{5}}{2} \right)^n \right]$$

```
1 from math import sqrt
2 n = int(input('n: '))
3 factor = 1/(1/2)(5)
4 operando1 = (1 + sqrt(5))/2
5 operando2 = (1 - sqrt(5))/2
6 fn = factor * (operando1**n - operando2**n)
7 print = fn
```


Ejercicio 18 - solución

- **Línea 2:** no se puede asignar a un literal; la variable debe llamarse `n`.
- **Línea 3:** en Python la yuxtaposición no es multiplicación; se necesita $1/\sqrt{5}$.
- **Línea 7:** `print = fn` *reemplaza* la función `print`; hay que **llamarla**.

Ejercicio 18 - solución

- **Línea 2:** no se puede asignar a un literal; la variable debe llamarse `n`.
- **Línea 3:** en Python la yuxtaposición no es multiplicación; se necesita $1/\sqrt{5}$.
- **Línea 7:** `print = fn` *reemplaza* la función `print`; hay que **llamarla**.

```
from math import sqrt
n = int(input('n: '))
factor = 1/sqrt(5)
operando1 = (1 + sqrt(5))/2
operando2 = (1 - sqrt(5))/2
fn = factor * (operando1**n - operando2**n)
print(fn)
```

Con `n = 10` imprime `55.000000000000014`: los float no son exactos, aunque $F_{10} = 55$.

Ejercicio 19 - propina

Un programa pide el costo de una comida y muestra el monto de la propina (15%) y el total a pagar.

Ingrese el costo de la comida: 50

Monto de la propina (15%): 7.5

Total a pagar: 57.5

Ejercicio 19 - propina

Un programa pide el costo de una comida y muestra el monto de la propina (15%) y el total a pagar.

Ingrese el costo de la comida: 50

Monto de la propina (15%): 7.5

Total a pagar: 57.5

Antes de escribir código:

- ¿Cuál es la información de **entrada** que conforma el estado inicial?
- ¿Cuál es la información de **salida** que da por resuelto el problema?
- ¿Cuáles son los **pasos** que llevan del estado inicial al estado final?

Ejercicio 19 - solución

```
costo = float(input('Ingrese el costo de la comida: '))  
propina = costo * 0.15  
total = costo + propina  
print('Monto de la propina (15%):', propina)  
print('Total a pagar:', total)
```

Ejercicio 20 - binario a decimal

Escriba un programa que transforme un número binario de 4 dígitos (formado sólo por 0 y 1) a su equivalente decimal.

Cada dígito representa una potencia de 2, de 2^3 a 2^0 . Por ejemplo, 1101 equivale a 13 porque

$$13 = 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0$$

Ingrese un número binario de 4 dígitos: 1101

El equivalente decimal es 13

Ejercicio 20 - solución

```
n = int(input('Ingrese un número binario de 4 dígitos: '))
d3 = n // 1000
d2 = (n // 100) % 10
d1 = (n // 10) % 10
d0 = n % 10
decimal = d3*2**3 + d2*2**2 + d1*2**1 + d0*2**0
print('El equivalente decimal es', decimal)
```

Ejercicio 20 - solución

```
n = int(input('Ingrese un número binario de 4 dígitos: '))
d3 = n // 1000
d2 = (n // 100) % 10
d1 = (n // 10) % 10
d0 = n % 10
decimal = d3*2**3 + d2*2**2 + d1*2**1 + d0*2**0
print('El equivalente decimal es', decimal)
```

Es el mismo truco del **Ejercicio 10**: // para descartar los dígitos de la derecha y % para quedarse con el último.

Ejercicio 21 - reloj analógico

El programa recibe la hora que marcan las manecillas de un reloj analógico y una cantidad de minutos, y muestra la hora que marcaría el reloj después de esos minutos.

Hora: 11

Minutos: 45

Minutos a avanzar: 30

El reloj marcará 12:15

Ejercicio 21 - solución

```
h = int(input('Hora: '))
m = int(input('Minutos: '))
avance = int(input('Minutos a avanzar: '))

total = m + avance
horas_extra = total // 60
# el -1 y el +1 son porque el reloj marca 12, no 0
h = (h - 1 + horas_extra) % 12 + 1
m = total % 60
print('El reloj marcará ' + str(h) + ':' + str(m))
```

Ejercicio 21 - solución

```
h = int(input('Hora: '))
m = int(input('Minutos: '))
avance = int(input('Minutos a avanzar: '))

total = m + avance
horas_extra = total // 60
# el -1 y el +1 son porque el reloj marca 12, no 0
h = (h - 1 + horas_extra) % 12 + 1
m = total % 60
print('El reloj marcará ' + str(h) + ':' + str(m))
```

Ojo: si los minutos quedan bajo 10, imprime 1:5 y no 1:05. Rellenar con ceros necesita

formato de strings, que veremos más adelante.

Ejercicio 22 - índice de masa corporal

El IMC es el peso en kilogramos dividido por el cuadrado de la estatura en metros.

Escriba un programa que lo calcule, considerando que recibe el **peso en libras** (1 libra = 0.45 kg) y la **estatura en centímetros**. Muestre el resultado con un decimal.

```
Peso [libras]: 154
```

```
Estatura [cm]: 175
```

```
Su IMC es 22.6
```

Ejercicio 22 - solución

```
libras = float(input('Peso [libras]: '))  
cm = float(input('Estatura [cm]: '))  
  
kg = libras * 0.45  
metros = cm / 100  
imc = kg / metros**2  
  
print('Su IMC es', round(imc, 1))
```

Ejercicio 22 - solución

```
libras = float(input('Peso [libras]: '))  
cm = float(input('Estatura [cm]: '))  
  
kg = libras * 0.45  
metros = cm / 100  
imc = kg / metros**2  
  
print('Su IMC es', round(imc, 1))
```

Las conversiones de unidades son **pasos intermedios**: conviene guardarlas en variables con nombre, no incrustarlas en una sola expresión gigante.

Ejercicio 23 - repartir gastos

Tres amigos compran distintas cosas para ver un partido y después quieren repartirse los gastos. El programa debe calcular cuánto debe cada uno.

Primer amigo: 4000

Segundo amigo: 2000

Tercer amigo: 6000

Primer amigo debe: 0.0

Segundo amigo debe: 2000.0

Tercer amigo debe: -2000.0

Ejercicio 23 - repartir gastos

Tres amigos compran distintas cosas para ver un partido y después quieren repartirse los gastos. El programa debe calcular cuánto debe cada uno.

Primer amigo: 4000

Segundo amigo: 2000

Tercer amigo: 6000

Primer amigo debe: 0.0

Segundo amigo debe: 2000.0

Tercer amigo debe: -2000.0

¿Qué significa que a un amigo le salga un valor **negativo**?

Ejercicio 23 - solución

```
a = float(input('Primer amigo: '))
b = float(input('Segundo amigo: '))
c = float(input('Tercer amigo: '))

cuota = (a + b + c) / 3

print('Primer amigo debe:', cuota - a)
print('Segundo amigo debe:', cuota - b)
print('Tercer amigo debe:', cuota - c)
```

Ejercicio 23 - solución

```
a = float(input('Primer amigo: '))  
b = float(input('Segundo amigo: '))  
c = float(input('Tercer amigo: '))  
  
cuota = (a + b + c) / 3  
  
print('Primer amigo debe:', cuota - a)  
print('Segundo amigo debe:', cuota - b)  
print('Tercer amigo debe:', cuota - c)
```

Un valor negativo significa que esa persona **pagó de más** y le deben devolver.

Ejercicio 24 - promedio aritmético y armónico

Calcule los dos promedios de tres valores. El **aritmético** es el de siempre; el **armónico** es

$$H = \frac{N}{\frac{1}{x_1} + \frac{1}{x_2} + \dots + \frac{1}{x_N}}$$

Valor 1: 2

Valor 2: 4

Valor 3: 8

Promedio aritmético: 4.666666666666667

Promedio armónico: 3.4285714285714284

Ejercicio 24 - solución

```
a = float(input('Valor 1: '))  
b = float(input('Valor 2: '))  
c = float(input('Valor 3: '))  
  
aritmético = (a + b + c) / 3  
armónico = 3 / (1/a + 1/b + 1/c)  
  
print('Promedio aritmético:', aritmético)  
print('Promedio armónico:', armónico)
```

Ejercicio 24 - solución

```
a = float(input('Valor 1: '))
b = float(input('Valor 2: '))
c = float(input('Valor 3: '))

aritmético = (a + b + c) / 3
armónico = 3 / (1/a + 1/b + 1/c)

print('Promedio aritmético:', aritmético)
print('Promedio armónico:', armónico)
```

El armónico **siempre** es menor o igual que el aritmético. Es el correcto para promediar velocidades: si voy a 30 km/h y vuelvo a 60 km/h, mi rapidez media es 40, no 45.

Ejercicio 25 - total de segundos

Al revés del **Ejercicio 6**: ahora recibimos horas, minutos y segundos, y hay que entregar el total de segundos.

Horas: 2

Minutos: 30

Segundos: 15

Son 9015 segundos en total

Ejercicio 25 - solución

```
horas = int(input('Horas: '))  
minutos = int(input('Minutos: '))  
segundos = int(input('Segundos: '))  
  
total = horas*3600 + minutos*60 + segundos  
print('Son', total, 'segundos en total')
```

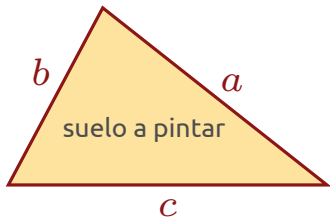
Ejercicio 25 - solución

```
horas = int(input('Horas: '))
minutos = int(input('Minutos: '))
segundos = int(input('Segundos: '))

total = horas*3600 + minutos*60 + segundos
print('Son', total, 'segundos en total')
```

Ida y vuelta: el Ejercicio 6 usa // y %; este usa * y +.

Ejercicio 26 - el problema del pintor



Un pintor necesita cubrir el suelo de una **habitación triangular**. Para estimar el costo del trabajo, debe calcular la superficie total a pintar. Como conocía algo del trabajo de Herón, sabía que debía guiarse por

$$A = \sqrt{s(s-a)(s-b)(s-c)}$$

donde $s = \frac{a+b+c}{2}$ es el semiperímetro.

Construya un programa en Python que resuelva el cálculo del pintor.

Ejercicio 26 - solución

```
from math import sqrt

a = float(input('Lado a [m]: '))
b = float(input('Lado b [m]: '))
c = float(input('Lado c [m]: '))

s = (a + b + c) / 2
area = sqrt(s*(s-a)*(s-b)*(s-c))

print('Superficie a pintar:', area, 'm2')
```

Ejercicio 26 - solución

```
from math import sqrt

a = float(input('Lado a [m]: '))
b = float(input('Lado b [m]: '))
c = float(input('Lado c [m]: '))

s = (a + b + c) / 2
area = sqrt(s*(s-a)*(s-b)*(s-c))

print('Superficie a pintar:', area, 'm2')
```

Con 5.5, 4.2 y 3.1 la superficie es 6.4666529209475865 m².

Ejercicio 26 - verificación

Verifique además que el cálculo anterior coincide con el valor entregado por

$$A = \frac{1}{4} \sqrt{(a^2 + b^2 + c^2)^2 - 2(a^4 + b^4 + c^4)}$$

```
area2 = sqrt((a**2+b**2+c**2)**2 - 2*(a**4+b**4+c**4)) / 4
print('Verificación:', area2)
print('¿Son iguales?', area == area2)
```

Ejercicio 26 - verificación

Verifique además que el cálculo anterior coincide con el valor entregado por

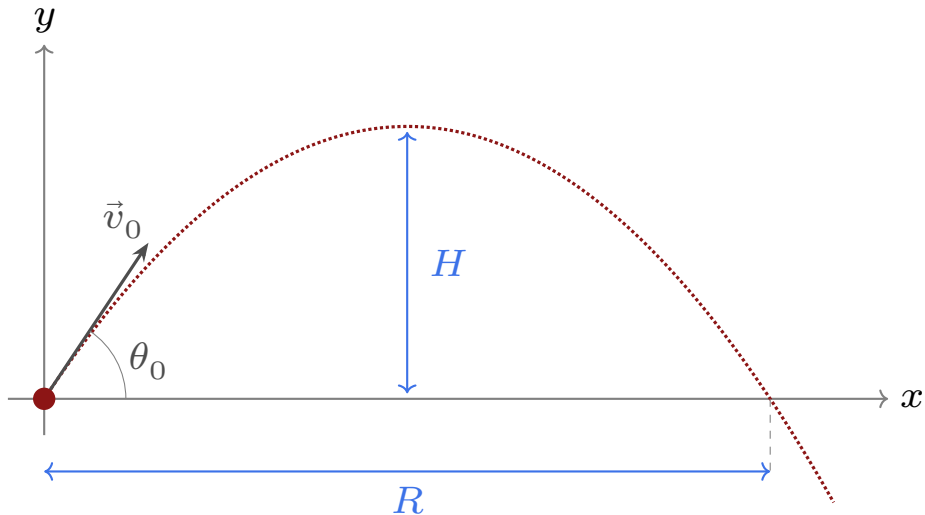
$$A = \frac{1}{4} \sqrt{(a^2 + b^2 + c^2)^2 - 2(a^4 + b^4 + c^4)}$$

```
area2 = sqrt((a**2+b**2+c**2)**2 - 2*(a**4+b**4+c**4)) / 4
print('Verificación:', area2)
print('¿Son iguales?', area == area2)
```

Con 5.5, 4.2 y 3.1 imprime **False**: las dos fórmulas difieren en 1.8×10^{-15} .

Los float **nunca** se comparan con `==`; se compara `abs(area - area2) < 1e-9`.

Ejercicio 27 - "La Guerra"



Ejercicio 27 - “La Guerra”

El movimiento de proyectiles se caracteriza por sus alcances máximos. Dados la rapidez inicial v_0 y el ángulo de lanzamiento θ_0 :

- **Altura máxima:** $H = \frac{v_0^2 \sin^2 \theta_0}{2g}$
- **Alcance horizontal:** $R = \frac{v_0^2 \sin(2\theta_0)}{g}$
- **Tiempo hasta la altura máxima:** $t_v = \frac{v_0 \sin \theta_0}{g}$

Construya un programa que calcule las tres cantidades. Identifique los datos de entrada, calcule según las fórmulas y reporte los valores con `print`.

Ejercicio 27 - solución

```
from math import sin, radians

v0 = float(input('Rapidez inicial [m/s]: '))
grados = float(input('Ángulo de lanzamiento [grados]: '))
g = 9.81

theta = radians(grados)  # ¡sin() trabaja en radianes!

H = (v0**2 * sin(theta)**2) / (2*g)
R = (v0**2 * sin(2*theta)) / g
tv = (v0 * sin(theta)) / g

print('Altura máxima:', H, 'm')
print('Alcance máximo:', R, 'm')
print('Tiempo hasta la altura máxima:', tv, 's')
```


Ejercicio 27 - el error clásico

Con $v_0 = 50 \text{ m/s}$ y $\theta_0 = 45^\circ$ debe salir:

Altura máxima: 63.71 m

Alcance máximo: 254.84 m

Tiempo hasta la altura máxima: 3.60 s

Ejercicio 27 - el error clásico

Con $v_0 = 50$ m/s y $\theta_0 = 45^\circ$ debe salir:

Altura máxima: 63.71 m

Alcance máximo: 254.84 m

Tiempo hasta la altura máxima: 3.60 s

Si se olvida `radians()` y se le pasa 45 directamente a `sin()`, Python **no da error**: calcula el seno de 45 *radianes* y entrega una altura de 92.26 m.

Un programa que corre no (necesariamente) es un programa correcto.

Ejercicio 28 - el mayor de 10 números

Para 10 números ingresados, encontrar el mayor valor.

`max()` recibe **dos** valores y entrega el más grande de los dos:

```
max(4, 9)    # entrega 9
```

¿Cómo se encuentra el mayor de diez, comparando de a dos?

Ejercicio 28 - solución

```
n = float(input('Número 1: '))
mayor = n

n = float(input('Número 2: '))
mayor = max(mayor, n)

n = float(input('Número 3: '))
mayor = max(mayor, n)

# ... y así hasta el número 10

print('El mayor es', mayor)
```

Ejercicio 28 - solución

```
n = float(input('Número 1: '))
mayor = n

n = float(input('Número 2: '))
mayor = max(mayor, n)

n = float(input('Número 3: '))
mayor = max(mayor, n)

# ... y así hasta el número 10

print('El mayor es', mayor)
```

La variable mayor va guardando **el mayor de los que se han leído hasta ahora**.

Ejercicio 28 - dos observaciones

1. ¿Por qué mayor parte valiando el primer número?

Si partiera en 0, con los diez números negativos el programa respondería 0, que no es ninguno de los ingresados.

Ejercicio 28 - dos observaciones

1. ¿Por qué mayor parte valiendo el primer número?

Si partiera en 0, con los diez números negativos el programa respondería 0, que no es ninguno de los ingresados.

2. Este par de líneas se repite nueve veces:

```
n = float(input('Número k: '))  
mayor = max(mayor, n)
```

Ejercicio 28 - dos observaciones

1. ¿Por qué mayor parte valiendo el primer número?

Si partiera en 0, con los diez números negativos el programa respondería 0, que no es ninguno de los ingresados.

2. Este par de líneas se repite nueve veces:

```
n = float(input('Número k: '))  
mayor = max(mayor, n)
```

¿Y si fueran 100 números? ¿Y si no supiéramos cuántos son?

Ese bloque repetido es exactamente el **cuerpo de un ciclo**, en la próxima unidad.

Para practicar

1. **Promedio de notas.** Pida el nombre de un estudiante y sus 3 notas, y entregue el promedio redondeado a un decimal.
2. **Costo de un viaje en auto.** Dada la distancia a recorrer en [km], el rendimiento del auto en [km/l] y el precio de la bencina, calcule el costo total.
3. **Cilindro.** Dados el radio de la base y la altura, muestre el área de una base, el área lateral, el área total y el volumen.
4. **Cuenta regresiva.** Pida una fecha objetivo y la fecha de hoy, y muestre cuántos días faltan (meses de 30 días).

aavelozb.github.io/py

